

ООП 2

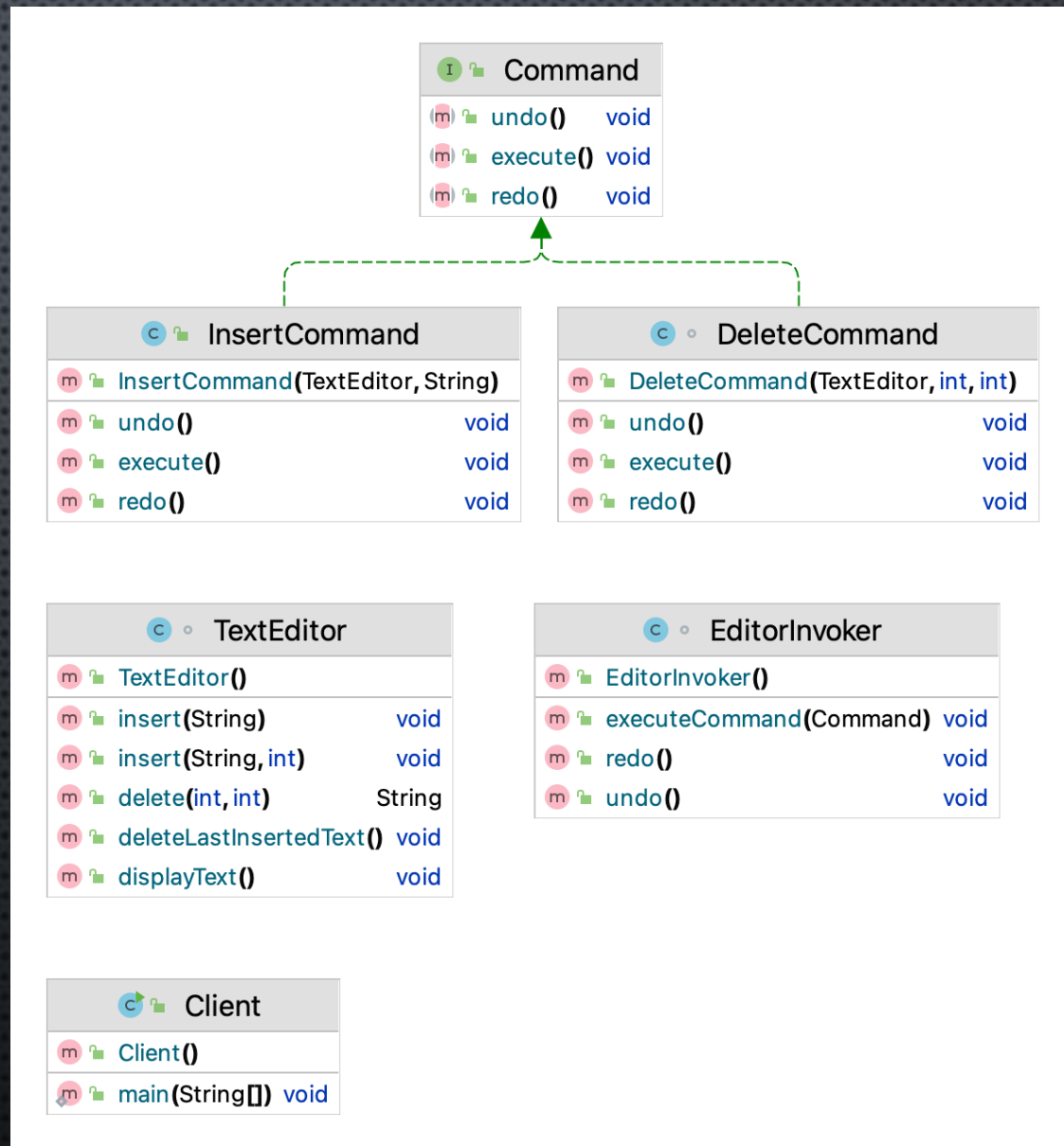
ПОВЕДЕНЧЕСКИ ШАБЛОНИ ЗА ДИЗАЙН 4 ЧАСТ

ПРОБЛЕМ

- ПОТРЕБИТЕЛСКИ ГРАФИЧЕН ИНТЕРФЕЙС
 - ИЗПЪЛНЕНИЕ НА КОМАНДИ
 - БЕЗ ЗНАНИЕ ЗА ДЕЙСТВИЕТО ИЛИ ЗА ИЗПЪЛНИТЕЛЯ

РЕШЕНИЕ

- COMMAND ШАБЛОН



COMMAND ШАБЛОН

```
PUBLIC INTERFACE COMMAND {  
    VOID EXECUTE();  
    VOID UNDO();  
}
```

COMMAND ШАБЛОН

```
CLASS TEXTEDITOR {  
    PRIVATE STRINGBUILDER TEXT = NEW STRINGBUILDER();  
  
    PUBLIC VOID INSERT(STRING S) {  
        TEXT.APPEND(S);  
    }  
}
```

```
    PUBLIC VOID DELETELAST(INT N) {  
        TEXT.DELETE(TEXT.LENGTH() - N, TEXT.LENGTH());  
    }  
  
    PUBLIC STRING GETTEXT() {  
        RETURN TEXT.TOSTRING();  
    }  
}
```

COMMAND ШАБЛОН

```
PUBLIC CLASS INSERTCOMMAND IMPLEMENTS COMMAND {  
    PRIVATE TEXTEDITOR EDITOR;  
    PRIVATE STRING TEXT;  
  
    PUBLIC INSERTTEXTCOMMAND(TEXTEDITOR EDITOR,  
        STRING TEXT) {  
        THIS.EDITOR = EDITOR;  
        THIS.TEXT = TEXT;  
    }  
  
    @Override  
    PUBLIC VOID EXECUTE() {  
        EDITOR.INSERT(TEXT);  
    }  
  
    @Override  
    PUBLIC VOID UNDO() {  
        EDITOR.DELETELAST(TEXT.LENGTH());  
    }  
}
```

COMMAND ШАБЛОН

```
PUBLIC CLASS COMMANDMANAGER {  
  
    PRIVATE STACK<COMMAND> UNDOSTACK = NEW STACK<>();  
    PRIVATE STACK<COMMAND> REDOSTACK = NEW STACK<>();
```

```
  
    PUBLIC VOID EXECUTECOMMAND(COMMAND CMD) {  
  
        CMD.EXECUTE();  
  
        UNDOSTACK.PUSH(CMD);  
  
        REDOSTACK.CLEAR();  
    }
```

```
  
    PUBLIC VOID UNDO() {  
  
        IF (!UNDOSTACK.ISEMPTY()) {  
  
            COMMAND CMD = UNDOSTACK.POP();
```

```
            CMD.UNDO();  
  
            REDOSTACK.PUSH(CMD);  
        }  
    }
```

```
  
    PUBLIC VOID REDO() {  
  
        IF (!REDOSTACK.ISEMPTY()) {  
  
            COMMAND CMD = REDOSTACK.POP();  
  
            CMD.EXECUTE();  
  
            UNDOSTACK.PUSH(CMD);  
        }  
    }  
}
```

COMMAND ШАБЛОН

```
PUBLIC CLASS DELETETEXTCOMMAND IMPLEMENTS COMMAND {  
  
    PRIVATE TEXTEDITOR EDITOR;  
  
    PRIVATE INT LENGTHTODELETE;  
  
    PRIVATE STRING DELETEDTEXT;  
  
    PUBLIC DELETETEXTCOMMAND(TEXTEDITOR EDITOR, INT  
LENGTHTODELETE) {  
  
        THIS.EDITOR = EDITOR;  
  
        THIS.LENGTHTODELETE = LENGTHTODELETE;  
    }  
  
    @OVERRIDE  
    PUBLIC VOID EXECUTE() {  
  
        STRING CURRENTTEXT = EDITOR.GETTEXT();
```

```
        IF (CURRENTTEXT.LENGTH() < LENGTHTODELETE) {  
            LENGTHTODELETE = CURRENTTEXT.LENGTH();  
        }  
  
        DELETEDTEXT =  
CURRENTTEXT.SUBSTRING(CURRENTTEXT.LENGTH() -  
LENGTHTODELETE);  
  
        EDITOR.DELETELAST(LENGTHTODELETE);  
    }  
  
    @OVERRIDE  
    PUBLIC VOID UNDO() {  
  
        EDITOR.INSERT(DELETEDTEXT);  
    }  
}
```

COMMAND ШАБЛОН

```
PUBLIC CLASS MAIN {  
    PUBLIC STATIC VOID MAIN(STRING[] ARGS) {  
        TEXTEDITOR EDITOR = NEW TEXTEDITOR();  
        COMMANDMANAGER MANAGER = NEW COMMANDMANAGER();  
        COMMAND INSERT = NEW INSERTTEXTCOMMAND(EDITOR, "HELLO WORLD!");  
        MANAGER.EXECUTECOMMAND(INSERT);  
        SYSTEM.OUT.PRINTLN("TEXT: " + EDITOR.GETTEXT()); // HELLO WORLD!  
        COMMAND DELETE = NEW DELETETEXTCOMMAND(EDITOR, 6);  
        MANAGER.EXECUTECOMMAND(DELETE);  
        SYSTEM.OUT.PRINTLN("AFTER DELETE: " + EDITOR.GETTEXT());  
        MANAGER.UNDO();  
        SYSTEM.OUT.PRINTLN("AFTER UNDO: " + EDITOR.GETTEXT());  
        MANAGER.REDO();  
        SYSTEM.OUT.PRINTLN("AFTER REDO: " + EDITOR.GETTEXT());  
    }  
}
```

Text: Hello World!
After delete: Hello
After undo: Hello World!
After redo: Hello

COMMAND ШАБЛОН

- ОТДЕЛЯ ОБЕКТА, КОЙТО ИЗВИКВА ОПЕРАЦИЯТА, ОТ ТОЗИ, КОЙТО ЗНАЕ КАК ДА Я ИЗПЪЛНИ
- КОМАНДИТЕ СА FIRST-CLASS ОБЕКТИ. МОГАТ ДА БЪДАТ МАНИПУЛИРАНИ И РАЗШИРЯВАНИ.
- КОМБИНАЦИЯ В КОМПОЗИТНА КОМАНДА
- ЛЕСНО ДОБАВЯНЕ НА НОВА КОМАНДА
 - НЕ Е НЕОБХОДИМО ДА СЕ ПРОМЕНЯТ КЛАСОВЕ

ПРОБЛЕМ

- ГРАФИЧЕН ИНТЕРФЕЙС
 - СЪСТОЯНИЕ НА ГРАФИЧНИ ЕЛЕМЕНТИ
 - МОДИФИКАЦИЯ
 - ВРЪЩАНЕ КЪМ ПРЕДХОДНО СЪСТОЯНИЕ - UNDO

РЕШЕНИЕ

- МЕМЕНТО ШАБЛОН
 - ПОЗВОЛЯВА ДА СЕ ЗАПАЗВА И ВЪЗСТАНОВЯВАТЕ ВЪТРЕШНОТО СЪСТОЯНИЕ НА ОБЕКТ, БЕЗ ДА СЕ РАЗКРИВАТ ПОДРОБНОСТИТЕ ЗА НЕГОВОТО ИЗПЪЛНЕНИЕ

МЕМЕНТО ШАБЛОН

- ORIGINATOR - ОБЕКТЪТ, ЧИЕТО СЪСТОЯНИЕ ТРЯБВА ДА БЪДЕ ЗАПАЗЕНО И ВЪЗСТАНОВЕНО
- МЕМЕНТО - ОБЕКТ, КОЙТО СЪХРАНЯВА СЪСТОЯНИЕТО НА ОБЕКТА ORIGINATOR.
- CARETAKER - ОТГОВАРЯ ЗА СЪХРАНЯВАНЕТО И УПРАВЛЕНИЕТО НА ОБЕКТИТЕ МЕМЕНТО.

MEMENTO ШАБЛОН

C • TextEditor	
m	TextEditor()
m	printText() void
m	save() Memento
m	restore(Memento) void
m	insert(String) void

C • Memento	
m	Memento(String)
f	state String
p	state String

C • History	
m	History()
m	push(Memento) void
m	pop() Memento

C • Client	
m	Client()
m	main(String[]) void

MEMENTO ШАБЛОН (ORIGINATOR)

```
CLASS TEXTEDITOR {  
    PRIVATE STRINGBUILDER TEXT;  
  
    PUBLIC TEXTEDITOR() {  
        THIS.TEXT = NEW STRINGBUILDER();  
    }  
  
    PUBLIC VOID INSERT(STRING CONTENT) {  
        TEXT.APPEND(CONTENT);  
    }  
}
```

```
    PUBLIC MEMENTO SAVE() {  
        RETURN NEW MEMENTO(TEXT.TOSTRING());  
    }  
  
    PUBLIC VOID RESTORE(MEMENTO MEMENTO) {  
        TEXT = NEW STRINGBUILDER(MEMENTO.GETSTATE());  
    }  
  
    PUBLIC VOID PRINTTEXT() {  
        SYSTEM.OUT.PRINTLN("CURRENT TEXT: " + TEXT);  
    }  
}
```

MEMENTO ШАБЛОН

```
CLASS MEMENTO {  
    PRIVATE STRING STATE;  
  
    PUBLIC MEMENTO(STRING STATE) {  
        THIS.STATE = STATE;  
    }  
  
    PUBLIC STRING GETSTATE() {  
        RETURN STATE;  
    }  
}
```

MEMENTO ШАБЛОН (CARETAKER)

```
IMPORT JAVA.UTIL.STACK;
```

```
CLASS HISTORY {  
    PRIVATE STACK<MEMENTO> MEMENTOSTACK;  
  
    PUBLIC HISTORY() {  
        MEMENTOSTACK = NEW STACK<>();  
    }  
  
    PUBLIC VOID PUSH(MEMENTO MEMENTO) {  
        MEMENTOSTACK.PUSH(MEMENTO);  
    }  
  
    PUBLIC MEMENTO POP() {  
        RETURN MEMENTOSTACK.POP();  
    }  
}
```

MEMENTO ШАБЛОН

```
PUBLIC CLASS CLIENT {  
    PUBLIC STATIC VOID MAIN(STRING[] ARGS) {  
        TEXTEDITOR TEXTEDITOR = NEW TEXTEDITOR();  
        HISTORY HISTORY = NEW HISTORY();  
  
        TEXTEDITOR.INSERT("HELLO");  
        HISTORY.PUSH(TEXTEDITOR.SAVE());  
  
        TEXTEDITOR.INSERT(" THERE!");  
        TEXTEDITOR.PRINTTEXT();  
  
        TEXTEDITOR.RESTORE(HISTORY.POP());  
        TEXTEDITOR.PRINTTEXT();  
    }  
}
```

Current Text: Hello There!
Current Text: Hello

МЕМЕНТО ШАБЛОН

- ЗАПАЗВА КАПСУЛАЦИЯТА НА ОБЕКТА
 - Оригиналния обект трябва да знае само за състоянието си, но състоянието трябва да се запазва извън него
- Може да струва ресурс